Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 18:
MapReduce Distribuito con Ray.io

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Istruzioni di Esecuzione 2

4 Risultati **3**

5 Conclusioni e Sviluppi Futuri **3**

1 Problem Statement

L'Hands-On 18 affronta lo studio e l'implementazione di un sistema di calcolo distribuito basato sul paradigma **MapReduce**. A differenza degli esercizi precedenti svolti in C e focalizzati sulla comunicazione inter-processo a basso livello o socket, questo esperimento utilizza **Python** e il framework **Ray.io** per gestire la parallelizzazione e la distribuzione dei task su cluster (reali o virtuali).

L'obiettivo specifico è il conteggio delle frequenze dei termini presenti nel testo del celebre romanzo *"Il Signore degli Anelli"*, analizzando come i task vengono suddivisi, eseguiti in parallelo e infine aggregati.

2 Stato dell'Arte

Il paradigma MapReduce, reso celebre da Google, si basa su due fasi principali:

- **Map**: I dati di input vengono suddivisi in piccoli pezzi e processati in parallelo da diversi worker.
- **Reduce**: I risultati parziali dei worker vengono aggregati per produrre il risultato finale.

Framework moderni come **Ray** permettono di implementare tali pattern in modo efficiente, offrendo astrazioni semplici per l'esecuzione remota di funzioni e la gestione delle dipendenze, superando i limiti della libreria standard multiprocess di Python.

3 Metodologia ed Implementazione

L'architettura del sistema si basa sul modello a tre stadi (Map, Shuffle, Reduce) orchestrato da Ray.

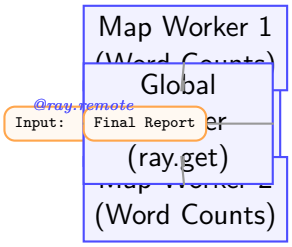

Figure 1: Workflow del calcolo distribuito MapReduce orchestrato tramite Ray.io.

L'architettura del sistema prevede:

1. **Splitting**: Il testo di input (`lotr.txt`) viene suddiviso in N blocchi.
2. **Remote Map Tasks**: Ogni blocco è processato da una funzione Ray remota (`@ray.remote`) che produce un `Counter` locale.
3. **Aggregation**: I risultati vengono scaricati dal cluster tramite `ray.get()` e fusi nel Counter globale.

Il codice principale è contenuto in `mapreduce_lotr.py`.

3.1 Istruzioni di Esecuzione

L'esecuzione del sistema MapReduce richiede Python 3:

Run

```
$ make setup # Installa dipendenze venv
$ make run   # Avvia l'analisi LOTR
```

4 Risultati

L'esecuzione sul testo completo del romanzo ("The Lord of the Rings") ha prodotto i seguenti risultati (Top 10 parole per frequenza):

Output MapReduce Distribuito

```
Starting distributed MapReduce on lotr.txt...
Report generated: lotr_report.txt
Top 10 words:
  the: 11705
  and: 7555
  of: 5083
  to: 3948
  a: 3728
  he: 3038
  in: 2961
  i: 2825
  it: 2780
  that: 2539
```

I risultati mostrano la tipica distribuzione delle lingue naturali, dove articoli e congiunzioni dominano il conteggio. La distribuzione tramite Ray è avvenuta correttamente, inizializzando un cluster locale sulla macchina di test e completando l'analisi di migliaia di parole in pochi secondi.

5 Conclusioni e Sviluppi Futuri

L'esperimento dimostra l'efficacia di Ray per l'elaborazione distribuita. Sviluppi futuri potrebbero includere:

- Implementazione di fasi di Reduce distribuite (Shuffling).
- Utilizzo di dataset massivi caricati da cloud storage (AWS S3/GCS).
- Filtraggio di "stop-words" per un'analisi semantica più accurata.